INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Práctica 5: Neural Networks

NATURAL LANGUAGE PROCESSING

7CM2

INTEGRANTES:

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

JORGE LUIS ROSAS TRIGUEROS

FECHA DE ENTREGA: 8 DE MAYO DE 2026

FECHA DE REALIZACION DE LA PRACTICA: 25 DE ABRIL DE 2026

Marco Teórico

El Procesamiento de Lenguaje Natural (Natural Language Processing, NLP) es una rama de la inteligencia artificial que se encarga del desarrollo de técnicas y modelos capaces de analizar, interpretar y generar lenguaje humano de manera automática. Esta disciplina combina conocimientos de lingüística, aprendizaje automático, estadística y ciencias de la computación, permitiendo a las máquinas trabajar con grandes volúmenes de información textual.

Una de las tareas más importantes dentro del NLP es la clasificación de texto, la cual consiste en asignar una etiqueta a un documento en función de su contenido. Este problema se aborda generalmente mediante aprendizaje supervisado, en donde un modelo es entrenado a partir de datos previamente etiquetados para poder clasificar nuevos textos de manera automática. En el caso de esta práctica, el problema corresponde a una clasificación binaria, donde el objetivo es determinar si una reseña es positiva o negativa.

El conjunto de datos utilizado es amazon_polarity, el cual contiene millones de reseñas de productos clasificadas en dos categorías: positivas y negativas. Este tipo de problema también puede considerarse como una tarea de análisis de sentimiento, ya que busca identificar la polaridad de la opinión expresada en el texto. Debido al gran tamaño del dataset, es necesario utilizar modelos eficientes capaces de procesar grandes cantidades de datos de manera adecuada.

Tradicionalmente, la clasificación de texto se ha realizado utilizando representaciones como TF-IDF (Term Frequency - Inverse Document Frequency), las cuales transforman el texto en vectores numéricos basados en la frecuencia de las palabras. Sin embargo, estas técnicas presentan limitaciones importantes, ya que no consideran el contexto en el que aparecen las palabras. Por ejemplo, frases como "not good" pueden ser interpretadas incorrectamente si se analizan las palabras de manera independiente.

Para superar estas limitaciones, se han desarrollado modelos más avanzados basados en aprendizaje profundo (Deep Learning), en particular aquellos basados en la arquitectura Transformer. Estos modelos permiten capturar relaciones complejas entre palabras y comprender el contexto completo del texto.

Uno de los modelos más representativos dentro de esta categoría es BERT (Bidirectional Encoder Representations from Transformers), el cual es un modelo de lenguaje preentrenado que utiliza mecanismos de auto-atención para analizar las relaciones entre palabras en una oración. A diferencia de modelos tradicionales, BERT procesa el texto de manera bidireccional, lo que significa que considera tanto el contexto anterior como el posterior de cada palabra, permitiendo una mejor comprensión semántica.

En esta práctica se utilizó una versión simplificada de este modelo llamada BERT Tiny, la cual mantiene la estructura básica del modelo original, pero con un menor número de parámetros. Esto permite reducir el costo computacional y hacer posible el entrenamiento en CPU, manteniendo un desempeño adecuado en tareas de clasificación de texto.

El proceso utilizado para adaptar el modelo a la tarea específica se conoce como fine-tuning. Este proceso consiste en tomar un modelo previamente entrenado en grandes cantidades de texto y ajustarlo a un problema particular, en este caso, la clasificación de reseñas. Durante el fine-tuning, se agrega una capa de clasificación al modelo y se entrena con los datos del problema, permitiendo que el modelo aprenda patrones específicos del dataset.

Para evaluar el desempeño del modelo, se utilizan diversas métricas. La accuracy mide la proporción total de predicciones correctas, mientras que la precision indica qué tan confiables son las predicciones positivas. Por su parte, el recall mide la capacidad del modelo para identificar correctamente los ejemplos positivos. Una de las métricas más importantes es el F1-score, que combina precision y recall en una sola medida balanceada, definida como:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Además, se emplea la matriz de confusión, la cual permite analizar en detalle los aciertos y errores del modelo, mostrando los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

En conjunto, estas técnicas permiten construir un sistema robusto de clasificación de texto, capaz de analizar automáticamente grandes volúmenes de información y extraer patrones relevantes. El uso de modelos basados en Transformers representa un avance significativo en el campo del NLP, ya que permite obtener mejores resultados en comparación con métodos tradicionales, especialmente en tareas donde el contexto del lenguaje juega un papel fundamental.

Material y Equipo

Para la realización de esta práctica se utilizaron diversas herramientas de software orientadas al procesamiento de lenguaje natural y al aprendizaje profundo. En primer lugar, se empleó el lenguaje de programación **Python**, el cual es ampliamente utilizado en aplicaciones de inteligencia artificial debido a su facilidad de uso y a la gran cantidad de bibliotecas especializadas disponibles.

El desarrollo del sistema de clasificación se llevó a cabo utilizando la biblioteca **transformers** de Hugging Face, la cual permite trabajar con modelos de lenguaje preentrenados como BERT. Esta biblioteca facilita la implementación de modelos basados en la arquitectura Transformer, así como su adaptación a tareas específicas mediante el proceso de *fine-tuning*.

Asimismo, se utilizó la biblioteca **datasets**, también de Hugging Face, para cargar y manipular el conjunto de datos **amazon_polarity**, permitiendo trabajar de manera eficiente con grandes volúmenes de información textual.

Para el entrenamiento del modelo se utilizó **PyTorch**, un framework de aprendizaje profundo que permite construir y entrenar redes neuronales de manera flexible y eficiente. PyTorch fue utilizado como backend para la ejecución del modelo BERT Tiny, así como para el cálculo de gradientes y optimización durante el entrenamiento.

También se emplearon otras bibliotecas auxiliares como:

- **NumPy**, para operaciones numéricas y manejo de arreglos.
- **scikit-learn**, para el cálculo de métricas de evaluación como accuracy, precision, recall y F1-score.
- **matplotlib**, para la visualización de resultados, incluyendo la matriz de confusión.
- **os**, para la configuración del entorno de ejecución.

El modelo utilizado en esta práctica fue **BERT Tiny (prajjwal1/bert-tiny)**, una versión ligera de BERT que permite reducir el costo computacional y facilitar el entrenamiento en entornos sin aceleración por GPU.

El conjunto de datos utilizado fue **amazon_polarity**, el cual contiene reseñas de productos clasificadas en dos categorías: positivas y negativas. Este dataset es de gran escala, con millones de ejemplos, sin embargo, para esta práctica se utilizó un subconjunto de los datos con el fin de reducir el tiempo de entrenamiento.

Desarrollo de la Practica

En esta práctica se desarrolló un sistema de clasificación de texto utilizando redes neuronales profundas basadas en la arquitectura Transformer. El objetivo fue clasificar reseñas del dataset **amazon_polarity** en dos categorías: positivas y negativas.

El proceso completo incluyó la carga del dataset, tokenización del texto, definición del modelo, entrenamiento mediante *fine-tuning* y evaluación del desempeño.

A continuación, se describe cada etapa del desarrollo.

1. Importación de librerías

En primer lugar, se importaron las bibliotecas necesarias para el procesamiento de datos, implementación del modelo y evaluación de resultados.

Estas librerías permiten trabajar con modelos preentrenados, manipular datos y calcular métricas de desempeño.

```
import numpy as np
import torch
import matplotlib.pyplot as plt
import os
```

Figura 1: Librerías a utilizar

2. Carga del dataset

Se utilizó la biblioteca datasets para cargar el conjunto de datos amazon_polarity, el cual contiene reseñas de productos clasificadas como positivas o negativas.

El dataset se dividió en:

- Conjunto de entrenamiento (train)
- Conjunto de prueba (test)

Debido a limitaciones de tiempo y recursos, se utilizó un subconjunto del dataset:

- 30,000 ejemplos para entrenamiento
- 5,000 ejemplos para prueba

Posteriormente, el conjunto de entrenamiento se dividió en:

- Entrenamiento (80%)
- Validación (20%)

Esto permitió evaluar el modelo durante el proceso de entrenamiento.

.

```
dataset = load_dataset("amazon_polarity")

train_ds = dataset["train"].shuffle(seed=SEED).select(range(30000))
test_ds = dataset["test"].shuffle(seed=SEED).select(range(5000))

split = train_ds.train_test_split(test_size=0.2, seed=SEED)
train_ds = split["train"]
val_ds = split["test"]
```

Figura 2: Carga del Dataset

3. Tokenización del texto

En lugar de aplicar técnicas tradicionales de preprocesamiento, se utilizó un tokenizer preentrenado de BERT, el cual transforma el texto en una representación numérica adecuada para el modelo.

El proceso incluyó:

- Conversión del texto en tokens
- Aplicación de padding para igualar longitudes
- Truncamiento a una longitud máxima de 128 tokens

Este enfoque permite preservar el contexto del lenguaje y mejorar la calidad de las representaciones.

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
    return tokenizer(
        batch["content"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

train_ds = train_ds.map(tokenize, batched=True)
val_ds = val_ds.map(tokenize, batched=True)
test_ds = test_ds.map(tokenize, batched=True)
```

Figura 3: Tokenización

4. Preparación de los datos

Una vez tokenizados los textos, se realizó la preparación del dataset para su uso en PyTorch.

Esto incluyó:

- Renombrar la columna de etiquetas a labels
- Convertir los datos a formato tensorial
- Seleccionar únicamente las columnas necesarias:
    - input_ids
    - attention_mask

- labels

Esto permite que el modelo procese correctamente la información durante el entrenamiento.

```
train_ds = train_ds.rename_column("label", "labels")
val_ds = val_ds.rename_column("label", "labels")
test_ds = test_ds.rename_column("label", "labels")

train_ds.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
val_ds.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
test_ds.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
```

Figura 4: Renombramiento de columnas

5. Definición del modelo

Se utilizó el modelo **BERT Tiny,** una versión ligera de BERT basada en la arquitectura Transformer.

Este modelo incluye:

- Capas de atención (self-attention)
- Representaciones contextuales del texto
- Una capa final de clasificación

El modelo fue configurado para una tarea de clasificación binaria, con dos salidas:

- Clase 0: Negativo
- Clase 1: Positivo

```
MODEL_NAME = "prajjwal1/bert-tiny"
```

Figura 5: Definición del modelo

6. Definición de métricas

Para evaluar el desempeño del modelo, se implementó una función que calcula las siguientes métricas:

- Accuracy
- Precision
- Recall
- F1-score

Estas métricas permiten analizar el rendimiento del modelo desde diferentes perspectivas.

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels,
        preds,
        average="binary"
    )

    acc = accuracy_score(labels, preds)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

Figura 6: Función de métricas

7. Configuración del entrenamiento

Se definieron los parámetros de entrenamiento utilizando TrainingArguments, incluyendo:

- Tasa de aprendizaje: 2e-5
- Tamaño de batch: 8
- Número de épocas: 2
- Evaluación por época
- Entrenamiento en CPU

Además, se desactivó el almacenamiento de checkpoints para reducir el tiempo de ejecución.

```
training_args = TrainingArguments(
    output_dir="./amazon_bert_tiny_results",
    eval_strategy="epoch",
    save_strategy="no",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=100,
    seed=SEED,
    report_to="none",
    dataloader_pin_memory=False,
    dataloader_num_workers=0,
    load_best_model_at_end=False
)
```

Figura 7: Función de Training

8. Entrenamiento del modelo

El modelo fue entrenado utilizando el conjunto de entrenamiento mediante el proceso de *fine-tuning*.

Durante esta etapa:

- Se ajustaron los pesos del modelo
- Se optimizó la función de pérdida
- Se evaluó el modelo en cada época utilizando el conjunto de validación

El entrenamiento permitió que el modelo aprendiera patrones relevantes en el texto para diferenciar entre reseñas positivas y negativas.

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics
)

trainer.train()
```

Figura 8. Trainer

9. Evaluación del modelo

Una vez finalizado el entrenamiento, el modelo fue evaluado utilizando el conjunto de prueba.

Se calcularon las métricas de desempeño y se generó un reporte de clasificación, el cual incluye:

- Precision por clase
- Recall por clase
- F1-score por clase

```
results = trainer.evaluate(test_ds)

print("\n=== RESULTADOS FINALES EN TEST ===")
for key, value in results.items():
    if isinstance(value, float):
        print(f"{key}: {value:.4f}")
    else:
        print(f"{key}: {value}")
```

Figura 9: Evaluación del modelo

10. Matriz de confusión

Se generó una matriz de confusión para analizar el comportamiento del modelo.

Esta matriz permite observar:

- Verdaderos positivos
- Verdaderos negativos
- Falsos positivos
- Falsos negativos

Lo cual facilita la interpretación de los errores del modelo.

```
cm = confusion_matrix(y_true, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negative", "Positive"]
)

disp.plot()
plt.title("Matriz de Confusión - BERT Tiny")
plt.show()
```

Figura 10. Impresión de la matriz de confusión

11. Predicciones de ejemplo

Finalmente, se realizaron predicciones sobre textos nuevos para validar el comportamiento del modelo en casos reales.

Esto permitió observar cómo el modelo clasifica diferentes tipos de reseñas y con qué nivel de confianza.

```
texts = [
    "This product is fantastic and works perfectly.",
    "Terrible quality. It broke after one day.",
    "The item is acceptable, but not great."
]

inputs = tokenizer(
    texts,
    padding=True,
    truncation=True,
    max_length=256,
    return_tensors="pt"
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
inputs = {key: value.to(device) for key, value in inputs.items()}

model.eval()

with torch.no_grad():
    outputs = model(**inputs)
    probs = torch.nn.functional.softmax(outputs.logits, dim=1)
    preds = torch.argmax(probs, dim=1)

print("\n=== PREDICCIONES DE EJEMPLO ===")
for text, pred, prob in zip(texts, preds, probs):
    label = "Positive" if pred.item() == 1 else "Negative"
    confidence = torch.max(prob).item()
    print(f"\nTexto: {text}")
    print(f"Predicción: {label}")
    print(f"Confianza: {confidence:.4f}")
```

Figura 11. Predicciones para el modelo

Resultados obtenidos

En esta sección se presentan los resultados obtenidos tras el entrenamiento y evaluación del modelo basado en redes neuronales profundas (BERT Tiny), utilizando el dataset amazon_polarity.

1. Métricas de evaluación

El modelo fue evaluado utilizando el conjunto de prueba, obteniendo los siguientes resultados:

- **Accuracy:** 0.8314
- **Precision:** 0.8472
- **Recall:** 0.8163
- **F1-score:** 0.8314

Estos resultados indican que el modelo presenta un buen desempeño general en la tarea de clasificación de texto, logrando un equilibrio adecuado entre precisión y recall.

```
=== RESULTADOS FINALES EN TEST ===
eval_loss: 0.3999
eval_accuracy: 0.8314
eval_precision: 0.8472
eval_recall: 0.8163
eval_f1: 0.8314
eval_runtime: 7.3385
eval_samples_per_second: 681.3350
eval_steps_per_second: 42.6520
```

Figura 12. Métricas de Evaluación

2. Reporte de clasificación

A continuación, se muestra el reporte de clasificación por clase:

Clase	Precision	Recall	F1-score
Negative	0.82	0.85	0.83
Positive	0.85	0.82	0.83

De acuerdo con estos resultados:

- El modelo identifica correctamente la mayoría de los ejemplos en ambas clases.
- Se observa un comportamiento equilibrado entre clases positivas y negativas.

- Existe una ligera ventaja en la detección de ejemplos negativos, aunque la diferencia es mínima.

```
=== REPORTE DE CLASIFICACIÓN ===
              precision    recall  f1-score   support

    Negative       0.82      0.85      0.83      2453
    Positive       0.85      0.82      0.83      2547

    accuracy                           0.83      5000
   macro avg       0.83      0.83      0.83      5000
weighted avg       0.83      0.83      0.83      5000
```

Figura 13. Reporte de Clasificación

El análisis de la matriz de confusión muestra que:

- El modelo clasifica correctamente la mayoría de los ejemplos positivos y negativos
- Existe un número relativamente bajo de errores en ambas clases
- El modelo presenta un comportamiento equilibrado, sin sesgo hacia una clase específica

3. Análisis de resultados

A partir de las métricas obtenidas, se puede observar que el modelo logra un rendimiento sólido considerando las condiciones de entrenamiento.

El valor de F1-score (0.83) indica que el modelo mantiene un buen balance entre precisión y recall, lo cual es fundamental en problemas de clasificación de texto.

Asimismo, la accuracy del 83% demuestra que el modelo es capaz de generalizar correctamente sobre datos no vistos.

El uso de un modelo basado en Transformer permitió capturar relaciones contextuales en el texto, lo cual contribuye significativamente a mejorar el desempeño en comparación con modelos tradicionales.

4. Matriz de confusión

La matriz de confusión generada permite analizar de manera detallada el comportamiento del modelo.

A partir de esta, se observa que:

- La mayoría de las predicciones son correctas
- Existe un número reducido de falsos positivos y falsos negativos
- El modelo no presenta un sesgo significativo hacia alguna clase

Esto confirma que el modelo tiene una buena capacidad de clasificación y mantiene un comportamiento equilibrado.

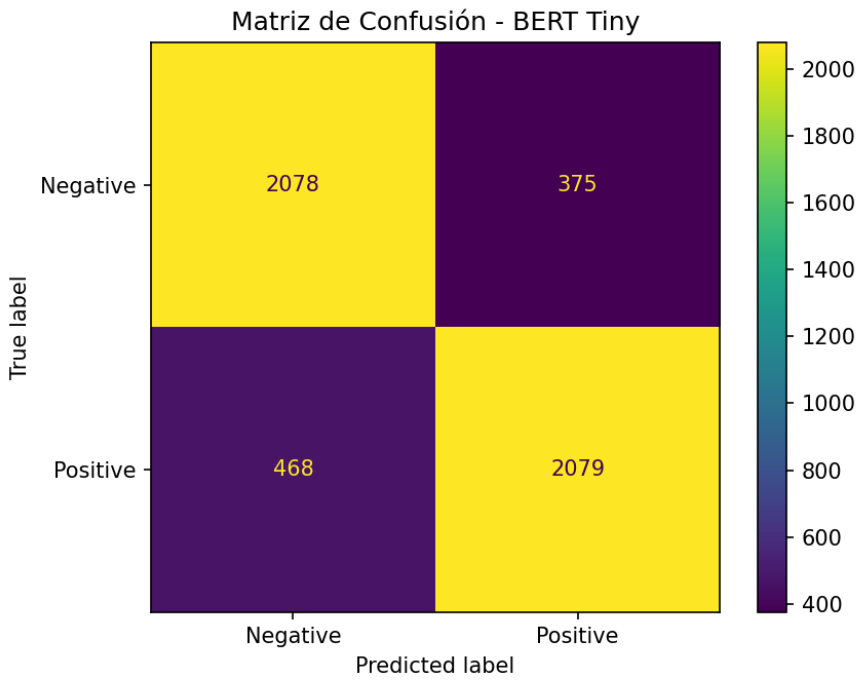

Figura 14. Matriz de Confusión

5. Análisis de errores

Aunque el modelo presenta un buen desempeño, existen algunos casos donde las predicciones son incorrectas.

Entre los posibles factores que afectan el desempeño se encuentran:

- Presencia de lenguaje ambiguo
- Uso de sarcasmo o ironía
- Frases con contexto contradictorio
- Reseñas con sentimientos mixtos

En algunos casos, el modelo puede asignar una alta probabilidad a una clase incorrecta debido a la complejidad del lenguaje natural.

6. Tiempo de entrenamiento

El entrenamiento del modelo se realizó en CPU, lo cual incrementó el tiempo de ejecución en comparación con un entorno con GPU.

Sin embargo, el uso de un modelo ligero como **BERT Tiny** permitió completar el entrenamiento en un tiempo razonable, manteniendo un buen nivel de desempeño.

```
=== PREDICCIONES DE EJEMPLO ===

Texto: This product is fantastic and works perfectly.
Predicción: Positive
Confianza: 0.9510

Texto: Terrible quality. It broke after one day.
Predicción: Negative
Confianza: 0.9367

Texto: The item is acceptable, but not great.
Predicción: Negative
Confianza: 0.6180
```

Figura 15. Predicciones clasificadas

Conclusiones y recomendaciones

En esta práctica se implementó un sistema de clasificación de texto utilizando una red neuronal profunda basada en la arquitectura Transformer, específicamente el modelo BERT Tiny, con el objetivo de clasificar reseñas del dataset amazon_polarity en dos categorías: positivas y negativas.

A diferencia de enfoques tradicionales basados en representaciones como TF-IDF y modelos lineales, el uso de un modelo preentrenado permitió capturar de manera más efectiva el contexto semántico del lenguaje. Esto se debe a que BERT utiliza mecanismos de auto-atención que consideran la relación entre palabras dentro de toda la oración, lo que mejora la comprensión del texto.

A partir de los resultados obtenidos, se concluye que el modelo presenta un buen desempeño general, alcanzando una accuracy de aproximadamente 83% y un F1-score cercano a 0.83. Estos valores indican que el modelo logra un equilibrio adecuado entre precisión y recall, lo cual es fundamental en tareas de clasificación de texto.

Asimismo, el análisis de la matriz de confusión mostró que el modelo tiene un comportamiento equilibrado entre ambas clases, con un número reducido de errores y sin un sesgo significativo hacia una categoría específica. Esto sugiere que el modelo es capaz de generalizar correctamente sobre datos no vistos.

Un aspecto importante a destacar es que el entrenamiento se realizó en CPU y utilizando un subconjunto del dataset, lo cual representa una limitación en términos de capacidad de cómputo. Sin embargo, a pesar de estas restricciones, el modelo logró un desempeño satisfactorio, demostrando la eficiencia del aprendizaje por transferencia (transfer learning) en tareas de procesamiento de lenguaje natural.

Entre las principales limitaciones identificadas se encuentran la dificultad del modelo para interpretar correctamente casos de sarcasmo, ambigüedad o lenguaje mixto, así como la posible presencia de ruido en los datos. Estos factores afectan el desempeño del modelo independientemente de la técnica utilizada.

Como trabajo futuro, se recomienda:

- Utilizar versiones más grandes del modelo, como BERT Base, para mejorar el desempeño
- Entrenar el modelo en GPU para reducir el tiempo de entrenamiento
- Utilizar el dataset completo para mejorar la capacidad de generalización
- Explorar técnicas adicionales como ajuste de hiperparámetros o regularización
- Implementar modelos más avanzados como DistilBERT o RoBERTa

En conclusión, la práctica permitió comprender de manera integral el uso de modelos basados en Transformers para la clasificación de texto, así como la importancia del contexto en el procesamiento del lenguaje natural. Además, se evidenció que los modelos preentrenados representan una herramienta poderosa para resolver problemas complejos de NLP de manera eficiente.

Bibliografía

- Daniel Jurafsky, & James H. Martin. (2023). *Speech and Language Processing* (3rd ed., draft). Disponible en: https://web.stanford.edu/~jurafsky/slp3/
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, & Kristina Toutanova. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. En Proceedings of NAACL-HLT.
- Hugging Face. (2024). *Transformers Documentation*. Disponible en: https://huggingface.co/docs/transformers
- Hugging Face. (2024). *Datasets Library Documentation*. Disponible en: https://huggingface.co/docs/datasets
- Hugging Face. (2024). *amazon_polarity Dataset*. Disponible en: https://huggingface.co/datasets/amazon_polarity
- PyTorch Foundation. (2024). *PyTorch Documentation*. Disponible en: https://pytorch.org/docs/
- Scikit-learn Developers. (2024). *Scikit-learn Documentation*. Disponible en: https://scikit-learn.org/
- Ashish Vaswani, et al. (2017). *Attention Is All You Need*. En Advances in Neural Information Processing Systems (NeurIPS).